CAbLECAR: efficiently scheduling QLDPC codes on a tileable spin qubit chip with shuttling

Jason D. Chadwick
Department of Computer Science
University of Chicago
Chicago, IL, USA
jchadwick@uchicago.edu

Frederic T. Chong
Department of Computer Science
University of Chicago
Chicago, IL, USA
chong@cs.uchicago.edu

***Abstract*—Semiconductor spin qubits are a promising platform for large-scale quantum computing, but have yet to take full advantage of the broad class of quantum low-density parity check (QLDPC) codes, which promise high encoding rates and efficient logic but require nonlocal connectivity between physical qubits. In this work, we investigate the implementation of QLDPC codes on a tileable, shuttling-based spin qubit architecture. By tailoring syndrome extraction circuits to the shuttling noise model, we significantly improve on previous surface code proposals and extend the feasible shuttling range of the architecture, enabling the implementation of more complex codes with long-range interactions. Taking inspiration from the field of robotics, we develop a coordinated shuttle scheduling algorithm that supports arbitrary codes and use it to benchmark the logical performance of a variety of promising code families. We find that the optimized schedules are up to 86% faster than hand-optimized schedules for certain code families. Through detailed circuit-level simulations, we identify specific QLDPC codes that improve upon prior surface code implementations by orders of magnitude, increasing encoding efficiency and reducing logical error rates. This work demonstrates the potential of shuttling-based spin qubit hardware platforms for scalable and efficient fault-tolerant quantum computation.**

***Index Terms*—quantum computing, quantum error correction, quantum LDPC codes, semiconductor spin qubits**

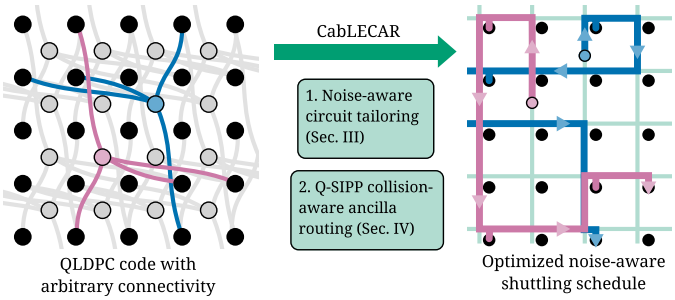

Fig. 1. Overview of the CAbLECAR framework. (Left) QLDPC codes demand complex, non-local parity checks, represented by arbitrary connections between data qubits (black) and ancilla qubits (colored). (Middle) To map these abstract requirements onto physical hardware, CAbLECAR applies noise-aware circuit tailoring to mitigate shuttling-induced dephasing and utilizes a novel optimization algorithm Q-SIPP to compute collision-free routing paths. (Right) The output is a hardware-compatible, time-optimized shuttling schedule where mobile ancilla qubits physically route along a tileable grid architecture to interact with stationary data qubits. Applied to efficient QLDPC codes, these optimized schedules yield orders of magnitude improvement in QEC performance over the surface code.

I. Introduction

The realization of large-scale fault-tolerant quantum computing (FTQC) requires robust quantum error correction (QEC) to protect fragile quantum information from decoherence and operational errors. The surface code has long been the leading candidate for FTQC due to its high threshold, strictly nearest-neighbor connectivity, and simple logical operations [1]–[4]. However, the surface code comes with a high physical qubit overhead to encode a single logical qubit, presenting a severe bottleneck for hardware scalability.

A promising alternative lies in the broader family of quantum low-density parity-check (QLDPC) codes. Many promising new QLDPC code families have been discovered that can achieve much higher encoding rates and lower logical error rates compared to the surface code [5]–[8]. Recent breakthroughs in methods for performing universal computation in QLDPC codes have further increased their abilities, allowing them to potentially be used not just for memory but for an entire quantum processor [9]–[18]. The primary barrier to implementing QLDPC codes on solid-state hardware is their requirement for complex, non-local connectivity between physical qubits—a feature that is native to modular atomic platforms but traditionally inaccessible to fixed two-dimensional solid-state arrays.

Semiconductor spin qubits offer a highly scalable pathway toward building millions of physical qubits, driven by their microscopic footprint and natural synergy with existing CMOS manufacturing processes [19]–[24]. Although recent hardware milestones have demonstrated steady improvements in both

This work is funded in part by the STAQ project under award NSF Phy-232580; in part by the US Department of Energy Office of Advanced Scientific Computing Research, Accelerated Research for Quantum Computing Program; and in part by the NSF Quantum Leap Challenge Institute for Hybrid Quantum Architectures and Networks (NSF Award 2016136), in part by the NSF National Virtual Quantum Laboratory program, in part based upon work supported by the U.S. Department of Energy, Office of Science, National Quantum Information Science Research Centers, and in part by the Army Research Office under Grant Number W911NF-23-1-0077. The views and conclusions contained in this document are those of the authors and should not be interpreted as representing the official policies, either expressed or implied, of the U.S. Government. The U.S. Government is authorized to reproduce and distribute reprints for Government purposes notwithstanding any copyright notation herein. FTC is the Chief Scientist for Quantum Software at Infleqtion.

array size [25]–[28] and operational fidelity [29]–[35], the realization of a dense monolithic grid of qubits remains an enormous engineering challenge. The strict requirements for control electronics, combined with severe wiring bottlenecks and signal crosstalk, force restrictive tradeoffs between array dimensions and control parallelism [21], [36].

To bypass the limitations of monolithic density, modern architectural proposals favor sparse, modular designs connected by coherent electron shuttling [37]–[39]. By physically transporting qubits across the chip to perform entangling operations, these architectures free up critical on-chip real estate for essential routing and wiring. Although prior studies have successfully mapped the standard surface code onto these shuttling-based platforms [40], demonstrating a viable path to near-term fault tolerance, such approaches are ultimately bottlenecked by the steep physical qubit overhead intrinsic to the surface code itself.

In this work, we demonstrate that the coherent shuttling capabilities of modern spin qubit architectures can be harnessed to unlock the benefits of QLDPC codes far exceeding the performance of the surface code. We present CAbLE-CAR (Coordinated Ancillae for LDPC codes with Efficient Collision-Aware Routing), a comprehensive framework for compiling and optimizing arbitrary QLDPC codes on a tiled, shuttling-based spin qubit processor (Figure 1). Recognizing that the shuttling range is physically limited by dephasing noise, we first introduce a noise-aware circuit tailoring technique that effectively cancels out shuttling-induced phase errors, dramatically extending the operational range of mobile ancilla qubits. To manage the complex web of non-local interactions in QLDPC codes, we adapt Safe Interval Path Planning (SIPP) from the field of autonomous robotics [41], yielding Q-SIPP: a scalable, collision-aware scheduling algorithm capable of routing ancilla qubits for arbitrary stabilizer check structures.

Specifically, the core contributions of this paper are as follows:

- We show how simple circuit-level tailoring can mitigate the dominant dephasing errors associated with coherent shuttling, improving the resilience of the baseline surface code to shuttling errors by 5-10× and enabling longer-distance shuttles for more efficient QEC.
- We develop Q-SIPP, a highly efficient routing algorithm that successfully coordinates collision-free paths for mobile ancilla qubits, achieving up to 85% reduction in shuttling overhead compared to hand-optimized schedules.
- We comprehensively benchmark the logical performance of several leading QLDPC code families, including tile, hypergraph-product-of-simplex, bivariate bicycle, and radial codes, against the baseline surface code on a shuttling-equipped architecture.
- Through detailed circuit-level noise simulations, we identify the specific hardware noise regimes where QLDPC codes begin to yield orders-of-magnitude improvements in logical error rate and physical qubit encoding efficiency.

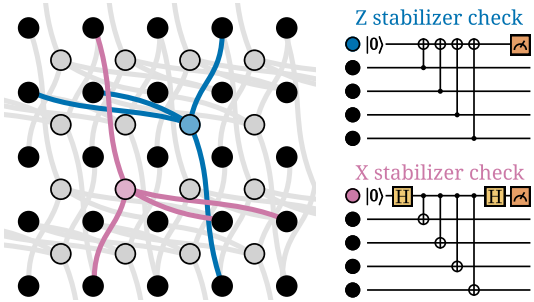

Fig. 2. Depiction of the structure of a quantum error correction code. A code block consists of a group of data qubits (black) and the logical code state is stabilized by repeated measurement of X and Z stabilizers. Typically, each stabilizer is measured using an ancilla qubit, which is initialized, entangled with the relevant data qubits, and then measured. Example X and Z checks are shown in pink and blue. The measurement results must then be decoded to determine whether any correction is needed to maintain an error-free code state. In this figure, X and Z checks both have weight 4, meaning that each ancilla qubit interacts with four data qubits in a syndrome extraction (SE) cycle.

II. Background

A. Quantum error correction

We briefly review the necessary fundamentals and terminology related to quantum error correction (QEC). For a more thorough background, we refer the reader to Refs. [1]–[3].

Quantum error correction is the process of encoding a small number of long-lived logical qubits in a large number of noisy physical qubits. In a standard QEC framework, a code block consists of two distinct types of qubit: data qubits, which collectively store the logical quantum state(s) and ancilla qubits, which are used to extract error information without collapsing the logical state. The logical code state is protected by enforcing a set of parity checks, which requires the repeated measurement of X and Z stabilizers. Figure 2 shows an abstract QEC code, with the data qubits (black) arranged in a grid and the ancilla qubits (gray and colored) amongst them. Each ancilla qubit is associated with an X or Z check, which is the measurement of the joint X or Z observable across a subset of the data qubits. Example X and Z checks are shown in pink and blue, along with the quantum circuits corresponding to one measurement of the stabilizers. An important parameter of these checks is their *weight*, which is the number of data qubits involved in the check. The weight of a code refers to the maximum weight of any individual parity check in the code.

The process of measuring these stabilizers is known as syndrome extraction (SE). During a typical syndrome extraction cycle, each ancilla qubit is initialized, entangled with the relevant data qubits, and then measured, as shown in the circuits in Figure 2. The measurement results must then be decoded with a classical decoding algorithm, typically in batches of d SE rounds, to determine whether any correction is needed to maintain an error-free code state.

The performance scaling of a QEC code can be quantified with the code distance d, which is the minimum number of physical faults that can cause a logical error. If all physical

errors occur with probability p, the expected logical error rate scales as $p_L \propto (p/p^*)^{(d+1)/2}$, where the parameter p^* is known as the code threshold and determines the maximum value of p such that increasing the code distance can arbitrarily suppress errors. QEC codes are often identified by their parameters $[[n, k, d]]$, where n is the number of data qubits in a code block and k is the number of logical qubits in the code block. For example, the surface code family consists of $[[d^2, 1, d]]$ codes. The quantity n/k is known as the encoding rate.

B. Semiconductor spin qubits

Semiconductor spin qubits have emerged as one of the most promising solid-state hardware platforms for large-scale quantum computation. These qubits typically encode quantum information in the spin states of individual electrons confined within semiconductor quantum dots. They offer compelling physical advantages for scalability, including a µm-scale footprint, the potential for operation at elevated temperatures, and direct compatibility with advanced CMOS manufacturing processes. Recent state-of-the-art demonstrations have shown the operation of two-dimensional arrays with capacities up to 16 qubits [25], [27], [28], and arrays with the capability to hold over 100 qubits have been successfully fabricated [42]. Gate and readout fidelities nearing or exceeding 99.9% have been demonstrated in a variety of hardware experiments [29]–[35]. T_1 relaxation times for spin qubits are generally extremely high, on the order of seconds [43]. T_2^* dephasing times are lower, on the order of hundreds of µs, but can be extended to T_2 times of tens of ms with dynamical decoupling techniques [44]. For a thorough review of semiconductor spin qubits, we refer the interested reader to Ref. [45].

Despite these physical advantages, the operation of dense, monolithic two-dimensional arrays of spin qubits is severely complicated by wiring bottlenecks, signal crosstalk, and the physical space required for control electronics. These challenges often force a difficult tradeoff between the width of the array and the control parallelism [21], [36]. To circumvent these dense connectivity challenges, recent architectural proposals have shifted toward modular, sparse arrays equipped with coherent qubit shuttling or long-range couplers [37]–[39]. In these designs, physical qubits are physically transported across the chip to perform entangling operations, leaving large areas of the chip available to allow for scalable wiring. Recently, a proof-of-principle weight-four parity check with a mobile ancilla qubit has been successfully demonstrated in silicon [46].

The hardware target we consider is heavily inspired by the SpinBus proposal by Kunne et al. [38]. The chip is built from unit cells as shown in Figure 3, where each unit cell contains one interaction zone (where two qubits can be entangled) and one readout zone (where a qubit can be initialized or measured). The unit cell is tiled to make a large array, with shuttling channels connecting the unit cells to each other. The crucial capability enabling this proposal, long-distance coherent shuttling, has been demonstrated in several test chips

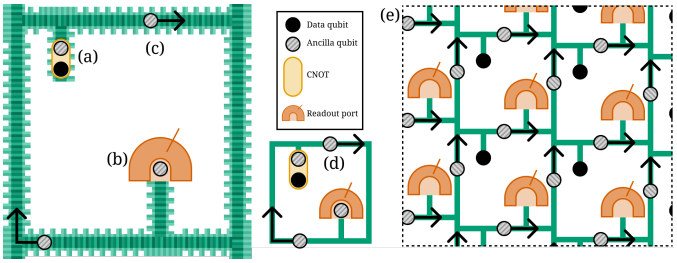

Fig. 3. The tiled unit cell architecture considered in this work. On the left is a closeup of one unit cell, which has (a) one interaction zone where gates are performed, (b) one readout port, where a qubit can be measured, and (c) shuttling channels, where qubits can travel within and between unit cells. (d) shows a simplified version of the unit cell, and (e) shows how the unit cells can be tiled into a grid-like arrangement.

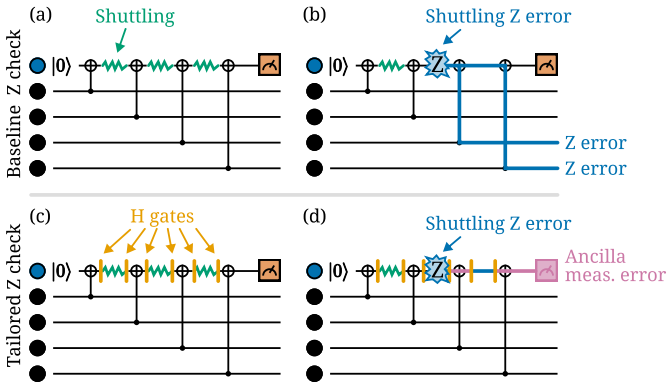

Fig. 4. Tailoring syndrome extraction circuits to mitigate shuttling noise. (a) Ancilla qubits are shuttled between CNOTs with data qubits. Shown here is a Z ancilla qubit, which is sensitive to shuttling errors. (b) Shuttling errors are heavily biased towards Z errors (dephasing). When such an error occurs, it propagates through the following CNOTs and manifests as one or more Z errors on the data qubits. (c) The tailored version of the circuit inserts H gates before and after each shuttling period. (d) This switches the Z error to an X error during each of the remaining CNOTs, preventing error propagation. The consequence is an X error on the ancilla qubit at the end of the circuit, which causes an incorrect measurement result.

[47]–[49]. In particular, shuttling an effective distance of $10\,\mu\mathrm{m}$, the expected edge length of one unit cell, was achieved with an overall fidelity of 99.5% and a speed of over $50\,\mathrm{m/s}$ [50]. Theoretical analysis predicts that these shuttling fidelities could reach as low as 10^{-5} (and speeds as high as hundreds of $\mathrm{m/s}$) through pulse shaping [51]. An important feature of coherent electron shuttling that we leverage in this work is that shuttling errors are heavily biased towards dephasing, with bit-flip errors occurring several orders of magnitude less frequently [50], [52].

III. Extending Shuttling Range with Noise-Aware Circuit Tailoring

We first describe a simple optimization to standard syndrome extraction circuits that greatly increases resilience to shuttling errors, enabling longer-distance stabilizer checks. We exploit the strong noise bias of the shuttling error channel, which is almost entirely dephasing errors [50], [52]. Inspecting the Z stabilizer syndrome extraction circuit in Figure 4a-b, we

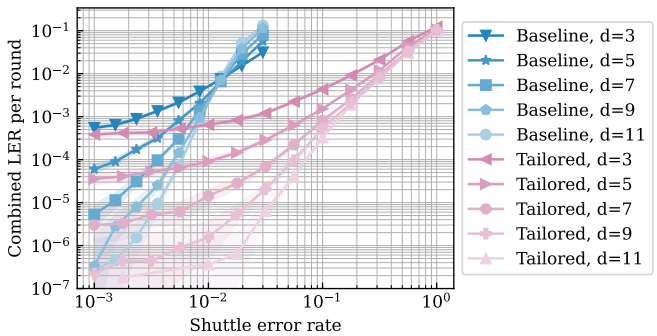

Fig. 5. Performance of the surface code shuttling schedule on the unit cell architecture [40] under varying shuttling noise, with and without CAbLECAR circuit tailoring. Tailoring allows the surface code to tolerate 5-10× higher shuttle error rates on the ancilla qubits. This increased noise resilience paves the way for considering more complex codes with longer shuttling paths.

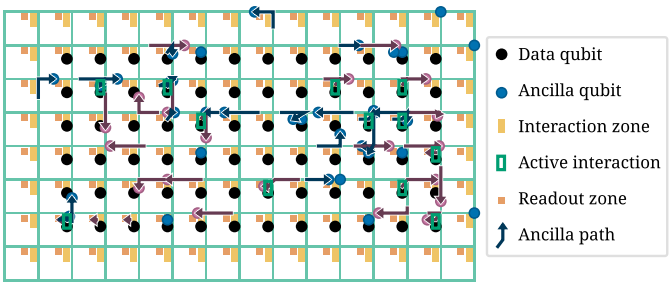

Fig. 6. Snapshot of optimized ancilla movement schedule for the [[72, 12, 6]] bicycle code. Data qubits (black) are in fixed positions, one per unit cell. Ancilla qubits (blue and pink) are shuttled between data qubits, performing CNOTs along the way. Colored arrows show ancilla movement over a 1μs time period.

see that a shuttling error on a Z ancilla qubit propagates to the data qubits that it interacts with afterwards, leading to one or more Z errors. Given that only the ancilla qubits are shuttled, we can mitigate this error propagation by simply appending Hadamard gates before and after every shuttling period. In the noiseless case, each pair of Hadamards is equivalent to the identity, so the syndrome extraction circuit still logically functions identically. As shown in Figure 4d, if a shuttling error occurs during a shuttling period, each Hadamard gate will toggle the error between a Z error and an X error. This prevents error propagation to the data qubits, instead causing an X error on the ancilla qubit, which becomes a bit-flip measurement error when the ancilla is measured. The X ancilla checks do not need to be modified because dephasing errors on X ancillae do not propagate to the data qubits.

This optimization can be immediately applied to the prior surface code schedule [40] to improve its resilience to shuttling errors. Figure 5 shows the combined X and Z memory error rate of surface codes of various distance as a function of the shuttling error rate with and without this circuit tailoring. Gate and measurement error rates are held constant at 10^{-3}. The bus error rate is the chance of a dephasing error per unit edge (in the surface code circuit, each ancilla qubit travels a total of four edge lengths per SE round). In the baseline circuit (blue) we observe a clear threshold around a bus error rate of 10^{-2}, and see that shuttling error rates closer to 10^{-3} are required to yield strong suppression of logical errors with increasing code distance. On the other hand, the tailored circuit (pink) can achieve similar logical error rates at a 5-10× higher bus error rate.

Because this circuit tailoring converts shuttling errors to ancilla measurement errors, increasing the temporal code distance by decoding over a window with more than the standard d SE rounds can further improve the per-round logical error rate. While this may allow the use of spatially-smaller codes to achieve the same logical performance, increasing the temporal distance of the code lengthens the logical clock cycle time, which may lead to a relative slowdown. These spacetime cost tradeoffs deserve a more in-depth exploration that is beyond the scope of this work; here we choose to simply decode over d SE rounds as usual and leave a more thorough study for future work.

We have shown that the shuttling noise circuit tailoring technique significantly improves the performance of the prior surface code schedule. Crucially, by relaxing the tolerable shuttling error threshold by nearly an order of magnitude, this tailoring technique physically permits the extended routing distances required by QLDPC codes. This widened error margin serves as the essential hardware foundation that makes the complex, long-range ancilla routing in CAbLECAR feasible.

IV. Quantum Safe Interval Path Planning (Q-SIPP) for Syndrome Extraction

Most prior proposals for the implementation of error correction codes with mobile qubits have resorted to hand-optimized movement patterns, exploiting the structure of particular code families that allow qubit movements to be synchronized [53]–[57]. However, this approach requires significant effort to apply to a new code family and is impractical for codes that have a less convenient structure. In this section, we describe a novel optimization approach that enables the implementation of arbitrary high-rate codes on shuttling-equipped architectures.

Scheduling the shuttling movements on our device takes the form of a simple problem: each ancilla qubit has a set of data qubits $\{d_i\}$ at fixed locations that it must visit. This list may either be strictly ordered to carry out a particular hook-error-avoiding syndrome extraction circuit, such as for the surface code, or may be unordered, allowing more freedom to choose the optimal ancilla order on the fly. For example, hypergraph product codes are known to allow any ordering of CNOTs in an SE circuit without a reduction in the effective code distance [58]. Upon arriving at a data qubit, the two qubits are entangled, and then the ancilla qubit either moves on to its next task or to a readout port if it has completed its tasks. The objective is to schedule shuttling operations to minimize the overall duration of the syndrome extraction round. Figure 6 shows a snapshot of a CAbLECAR-optimized schedule for

the [[72,12,6]] bicycle code [5], where we show the paths of ancilla qubits (pink and blue) over a 1 µs time interval.

The ancilla qubits are scheduled one-by-one in a predefined order, where each previously-scheduled ancilla qubit is treated as a dynamic obstacle that must be avoided. This leads to a problem space where each device component (shuttle channel, interaction zone, or readout port) has time-dependent availability. Standard path planning in dynamic environments often suffers from an explosion in the search space because it must add time as a fully discretized, independent dimension. To avoid this, CAbLECAR employs a quantum-tailored variant of Safe Interval Path Planning (SIPP) [41] that we refer to as Quantum-SIPP (Q-SIPP).

SIPP is based on the observation that the number of distinct time intervals where each component is available (unoccupied) is finite and generally very small. Therefore, instead of fully discretizing time, we can simply keep track of *safe intervals* for each component. A safe interval is defined as a time period where a component is unoccupied, bounded by occupations on either side, unless it is the final interval which may extend to infinity. By representing states with safe intervals rather than discrete timesteps, the search space is significantly compressed, allowing for highly efficient planning [41]. Each Q-SIPP schedule optimization performed in the preparation of this manuscript, for codes with up to 512 data qubits and 494 ancilla qubits, ran in under 5 minutes on a single CPU core.

A. State Space Formulation

To accommodate the specific rules of the quantum hardware, the standard SIPP configuration space is expanded. A state S in Q-SIPP is defined as the tuple $\langle loc, comp, i, T\rangle$:

- loc: The spatial coordinates of the qubit on the chip.
- $comp$: The currently-occupied hardware component type (e.g. shuttle channel, interaction zone, readout).
- i: The index of the safe interval for this specific location and component.
- T: A boolean tuple tracking task progress (specifically, which data qubits the ancilla has successfully interacted with).

By tracking time with safe intervals, we do not need to record the precise time that the qubit arrives at or leaves a location, significantly reducing problem complexity. A compiled shuttling schedule with exact timings can be extracted from the final solution path.

B. A* Search Loop

Algorithm 1 outlines the core A* search operating over these states. It evaluates nodes based on the objective function $f(s) = g(s) + h(s)$, where $g(s)$ tracks the cost of the shortest known path from the start state to s and the heuristic function $h(s)$ estimates the remaining cost to task completion. At each step, the state s with the minimum objective function is removed from the $OPEN$ set, which is implemented as a priority queue. The search successfully terminates when it expands a goal state, which is a state where all tasks in T are complete and the qubit is safely resting in a READOUT component. SIPP guarantees that this schedule will be time-optimal, given fixed preexisting safe interval constraints. If the chosen state is not a goal state, Q-SIPP finds all valid successor states of s using the GETSUCCESSORS function described in the following subsection. These states are added to the $OPEN$ set.

Algorithm 1 Q-SIPP Search

Require: Start state s_{start}, goal conditions
Ensure: Optimal time-ordered schedule

```
g(s_start) ← 0; OPEN ← {s_start}
while OPEN ≠ ∅ do
    s ← arg min_{s'∈OPEN} f(s')
    OPEN ← OPEN \ {s}
    if IsComplete(s.T) and s.comp = READOUT then
        return TracePath(s)
    end if
    for each s' ∈ GetSuccessors(s) do
        t_arr ← max(g(s) + c(s, s'), s'.i_start)
        if t_arr < g(s') then
            g(s') ← t_arr
            f(s') ← g(s') + Heuristic(s')
            insert or update s' in OPEN with f(s')
        end if
    end for
end while
return failure
```

Algorithm 2 GETSUCCESSORS(s)

Require: Current state $s = \langle loc, comp, i, T\rangle$

```
succ ← ∅
P ← PendingDataLocations(s.T)
if s.comp = INTERSECTION then
    succ ← succ ∪ GetValidShuttles(s)
end if
for each c ∈ PossibleComponents \ {s.comp} do
    succ ← succ ∪ GetValidDisplacements(s, c)
end for
if s.comp = INTERACTION and s.loc ∈ P then
    if s.i_end ≥ g(s) + t_CX then
        T' ← UpdateTask(s.T, s.loc)
        s' ← ⟨s.loc, s.comp, s.i, T'⟩
        insert s' into succ with cost f(s') = f(s) + t_CX
    end if
end if
return succ
```

C. Successor Generation and Transitions

Algorithm 2 determines how the successor states are chosen upon expansion of a state. There are several actions an ancilla qubit can take to transition to a new state:

1) **Shuttle:** If the current state's component is an INTERSECTION, the qubit can shuttle to an adjacent INTERSECTION. This transition is valid if the intermediate

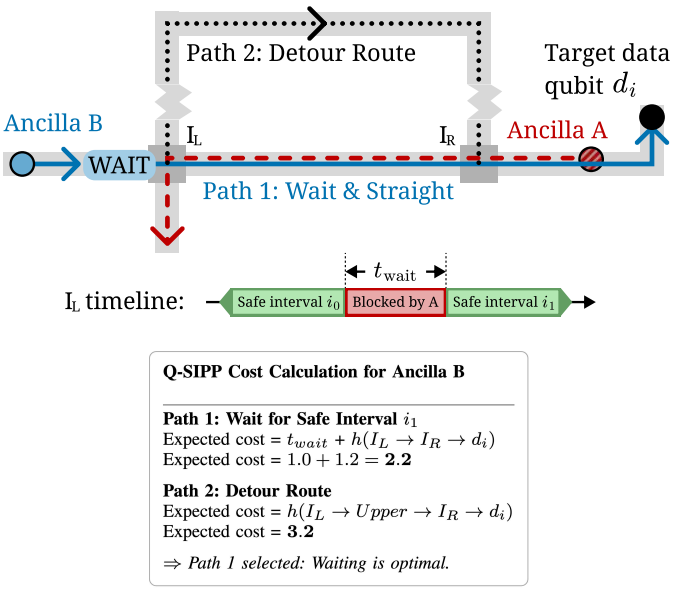

Fig. 7. A visualization of the Q-SIPP scheduling process resolving a spatial collision. Ancilla B must route to d_i but its shortest path through I_L is temporarily blocked by Ancilla A. Instead of forcing B to take a collision-free detour, Q-SIPP projects the timeline of I_L into continuous safe intervals. By calculating the heuristic cost $h(s)$ of the detour versus the known penalty t_{wait} to utilize the next available safe interval (i_1), the algorithm correctly determines that waiting at I_L yields the time-optimal schedule.

CHANNEL and destination INTERSECTION have compatible safe intervals, meaning that the ancilla qubit can leave the current INTERSECTION, shuttle through the CHANNEL, and arrive at the new INTERSECTION during these safe intervals. There may be multiple distinct transition states if the destination INTERSECTION has multiple reachable safe intervals.

2) **Displace:** Transition between hardware component layers at its current location (e.g., moving from an INTERSECTION into an INTERACTIONZONE or READOUTZONE).
3) **Execute Gate:** Perform a CX gate with a data qubit, if resting in an INTERACTIONZONE corresponding to a pending data qubit, resulting in a new state with updated task progress tracker T.

Idling is handled implicitly via the safe interval formalism and does not need to be directly encoded as a state transition.

D. Heuristic function

To ensure that the A* search remains highly directed, Algorithm 3 provides an admissible heuristic that estimates the remaining cost-to-goal. For a given state, the heuristic function identifies all pending data qubits remaining in the task tuple T. It then utilizes a traveling salesman solver to determine the shortest possible path to complete the remaining tasks, ignoring dynamic obstacles entirely. The overall estimated cost is the total duration of shuttling along this path plus the durations of performing the required displacements and CX gates at each stop.

Algorithm 3 HEURISTIC(s)

Require: Current state s
Ensure: Estimated minimum remaining time

```
P ← PendingDataLocations(s.T)
if P = ∅ then                                   ▷ All tasks complete
    if s.comp = READOUT then
        return 0
    else
        return t_displace
    end if
end if
cost ← 0
cur ← s.loc
if s.comp = INTERACTION and cur ∈ P then
    cost ← cost + t_CX + t_displace
    P ← P \ {cur}
end if
if s.comp ≠ INTERACTION and cur ∉ P then
    cost ← cost + t_displace
end if
π ← SOLVETSP(cur, P)
for each segment (u, v) ∈ π do
    cost ← cost + DIST(u, v) · t_shuttle + t_CX + 2 · t_displace
end for
return cost
```

The resulting heuristic cost dynamically incorporates the minimum required shuttling time between the remaining data qubits, the hardware-specific displacement delays required to interact with them, and the strict durations of the remaining quantum gates. Because this heuristic always gives the shortest possible collision-free time to the goal, it is consistent and satisfies the triangle inequality, so the completeness claims from Ref. [41] hold. The codes we evaluate in this work all have check weight ≤ 8, for which the exact TSP solver runs extremely quickly. For codes with a sufficiently-large stabilizer check weight, heuristic TSP solvers may be required instead of exact methods, which would remove the completeness guarantee, but Q-SIPP could still generate valid solution schedules. Although we have not exhaustively mapped its scalability bounds, preliminary results show that Q-SIPP successfully generates high-quality schedules for check weights up to 12 when paired with approximate TSP solvers.

E. Extensions

The Q-SIPP algorithm is quite flexible and can easily be extended to consider alternative problem formulations. For example, we make a small modification to account for the circuit tailoring described in Section III by replacing each CX gate with a sequence of Hs and CXs (for Z ancilla qubits only). This can be incorporated by simply changing t_{CX} to $t_{CX} + 2t_H$ in Algorithms 2 and 3.

Another key feature of the Q-SIPP algorithm is its adaptability to different hardware layouts. For example, the original SpinBus proposal [38] involves a hexagonal grid of unit cells

Code type	$[[n, k, d]]$	n/k	Check weight	Source
Surface	[[9, 1, 3]]	9.0	4	[59]
Surface	[[25, 1, 5]]	25.0	4	[59]
Surface	[[49, 1, 7]]	49.0	4	[59]
Surface	[[81, 1, 9]]	81.0	4	[59]
Surface	[[121, 1, 11]]	121.0	4	[59]
Tile	[[288, 8, 12]]	36.0	6	[7]
Tile	[[288, 8, 14]]	36.0	8	[7]
Tile	[[288, 18, 13]]	16.0	8	[7]
Tile	[[512, 18, 19]]	28.4	8	[7]
Simplex	[[98, 18, 4]]	5.4	6	[15]
Simplex	[[450, 32, 8]]	14.1	6	[15]
BB	[[72, 12, 6]]	6.0	6	[5]
BB	[[90, 8, 10]]	11.3	6	[5]
BB	[[108, 8, 10]]	13.5	6	[5]
BB	[[144, 12, 12]]	12.0	6	[5]
BB	[[288, 12, 18]]	24.0	6	[5]
Radial	[[90, 8, 10]]	11.3	6	[6]
Radial	[[144, 8, 12]]	18.0	6	[6]
Radial	[[198, 8, 14]]	24.8	6	[6]
Radial	[[192, 18, 12]]	10.7	8	[6]
Radial	[[224, 18, 14]]	12.4	8	[6]
Radial	[[256, 18, 16]]	14.2	8	[6]
Radial	[[352, 18, 20]]	19.6	8	[6]

TABLE I
CODE CONSTRUCTIONS STUDIED IN THIS WORK

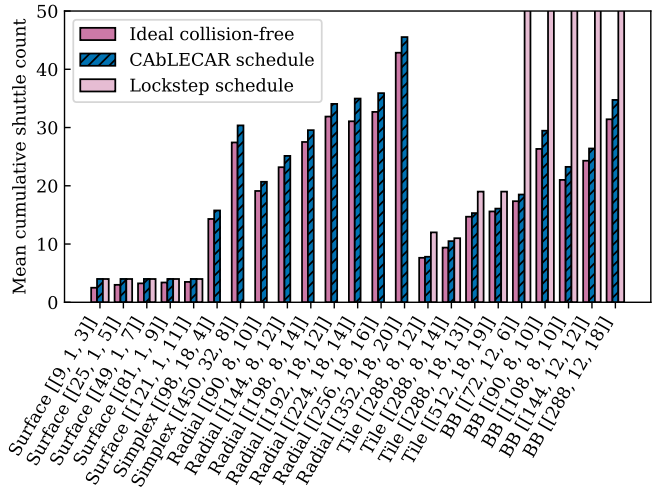

Fig. 8. Comparing average shuttle distance for CAbLECAR schedules compared to ideal minimum shuttle counts (assuming ancilla qubits can move freely past each other without collisions). We also compare to a "lockstep" schedule (all ancilla qubits moving together in a pattern) for codes where such a schedule exists. We find that CAbLECAR outperforms all lockstep schedules and is close to the collision-free ideal, with a minor 12.5% overhead (geomean). CAbLECAR reduces shuttle cost for Tile and BB codes by 19% and 86%, respectively, compared to hand-optimized lockstep schedules.

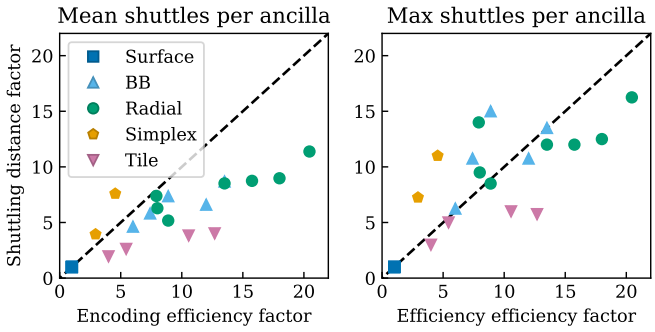

Fig. 9. Comparing shuttling overhead to efficiency factor kd^2/n. Higher-efficiency codes from the Radial and Tile families improve encoding efficiency by more than their increase in shuttle count.

instead of the square grid that we consider in this work. Alternatively, due to wiring and chip layout constraints, it may be beneficial to adopt a sparser layout with dense clusters of data qubits connected by long-range shuttling channels. In either case, Q-SIPP can be easily modified to optimize over arbitrary shuttling connectivities.

V. COMPARING CODE PERFORMANCE

We choose several promising families of QLDPC codes to compile with CAbLECAR and analyze. The chosen code instances are summarized in Table I. In addition to the surface code, which we use as a baseline, we study the following code families: **tile codes**, which maintain surface-code-like locality but offer improved encoding efficiency [7]; **hypergraph product codes of simplex codes**, which offer compelling automorphism-based logical operations [15], [18]; high-threshold and low check weight **bivariate bicycle codes** [5]; and **radial codes**, which offer compelling performance and single-shot decodability [6], but do not easily translate to hardware implementations due to their more complex check structures. The table lists each code with its key parameters $[[n, k, d]]$, where n is the number of data qubits in a code block, k is the number of encoded logical qubits, and d is the code distance. The encoding rate n/k is therefore the number of physical qubits required to encode each logical qubit.

Scheduling these codes with CAbLECAR is as simple as specifying the X and Z checks and the fixed positions of the data qubits. For the data qubit layout, we use the standard canonical rectangular layouts for the surface, tile, BB, and simplex codes. For radial codes and other codes without canonical layouts, we place the data qubits in a $\sqrt{n} \times \sqrt{n}$ square grid where data qubit i is mapped to coordinates $\left(i \bmod \sqrt{n}, \lfloor i/\sqrt{n} \rfloor\right)$. It is interesting future work to consider optimizing the placement of these data qubits to further improve scheduling.

A. Shuttle distance comparison

Figure 8 shows the average number of unit-edge shuttles per ancilla qubit across the studied code instances. We compare the CAbLECAR optimized schedules against two baselines. First, the "ideal collision-free" schedule assumes that shuttled qubits are allowed to occupy the same shuttling channel and even pass by one another without collisions. Compared to this baseline, the CAbLECAR schedules (which prevent simultaneous occupation of any shuttling channel) incur a geomean 12.5% shuttling overhead, demonstrating surprisingly near-

Operation	Duration	Error rate	Noise structure
CX gate	100 ns	10^{-3}	Depolarizing
H gate	100 ns	10^{-3}	Depolarizing
Initialization	500 ns	10^{-3}	Bit-flip (X)
Measurement	500 ns	10^{-3}	Bit-flip (X)
Shuttle (one edge)	1000 ns	$10^{-4} - 10^{-2}$	Phase-flip (Z)
Displace	200 ns	$10^{-4} - 10^{-2}$	Phase-flip (Z)
Idle decoherence	t	$1 - e^{-t/T_1}$	Bit-flip (X)
Idle dephasing	t	$1 - e^{-t/T_2}$	Phase-flip (Z)

TABLE II
OPERATIONS AND ASSOCIATED ERROR RATES FOR CIRCUIT-LEVEL SIMULATION

ideal shuttling counts. For the Tile and BB code families, we also compare to "lockstep" schedules, in which all X and Z ancilla qubits move together synchronously [54]. These code families admit lockstep scheduling because the X and Z checks are translationally identical. CAbLECAR outperforms these lockstep schedules by geomean 19% and 86%, respectively. For Tile codes, the intuitive reason for the improvement is that the X and Z checks on the edges of the array are incomplete, yet their ancilla qubits perform the same number of shuttles as the bulk checks, raising the average shuttle count. Similar edge effects lead to the large lockstep shuttle count for BB codes, but in this case the shuttle count is much higher because checks near the edge wrap around to the opposite side.

In Figure 9, we plot the relative shuttles per ancilla qubit in CAbLECAR schedules compared to the surface code, shown against the encoding efficiency factor kd^2/n. This metric is a convenient way to express the overall improvement that a QLDPC code achieves over the surface code (which has $kd^2/n = 1$ for all d), either by improving the encoding rate k/n or by increasing the code distance d. We see in Figure 9 that most of the studied codes achieve a high efficiency factor with only a modest increase in the mean relative shuttling distance, with Tile codes seeming to give the best tradeoff. Although the maximum shuttle counts are much higher, we expect the mean to be more representative of the impact on the logical error rate.

B. Logical performance

However, while kd^2/n can be a useful overall metric when considering long-term scalability, the code distance d does not fully capture the logical performance of a code in finite, realistic noise regimes. We therefore use Stim [60] to perform circuit-level simulations of logical memory experiments for the studied code instances. For each code, we convert the compiled CAbLECAR schedule for d rounds of syndrome extraction into a Stim circuit, adding error instructions after each initialization, gate, shuttle sequence, or readout. We also add idling errors on qubits during inactive periods, such as while a data qubit waits between ancilla visits. The operations and associated error rates are summarized in Table II. We use constant $T_1 = 10\,\mathrm{s}$ and $T_2 = 10\,\mathrm{ms}$ (assuming dynamical decoupling techniques are used during any idle periods).

We perform simulations with all error rates constant except the shuttling and displacement error rates, which we vary between 10^{-3} and 10^{-1}. We use the noise-aware circuit tailoring technique described in Section III for all schedules, including the surface code. For some codes, the logical error rates were too low to measure directly with Monte Carlo simulation methods, so we perform fits to the data for higher p_{sh} and extrapolate to lower p_{sh} as needed. We fit the theoretical model $p_L = A(b + c \cdot p_{\mathrm{sh}})^{d_{\mathrm{eff}}}$, where A, b, c, and d_{eff} are fit parameters. Figure 10 shows the simulation data and the obtained fits. For most codes, we see diminishing returns beyond $p_{\mathrm{sh}} \approx 10^{-3}$, as gate errors begin to limit performance.

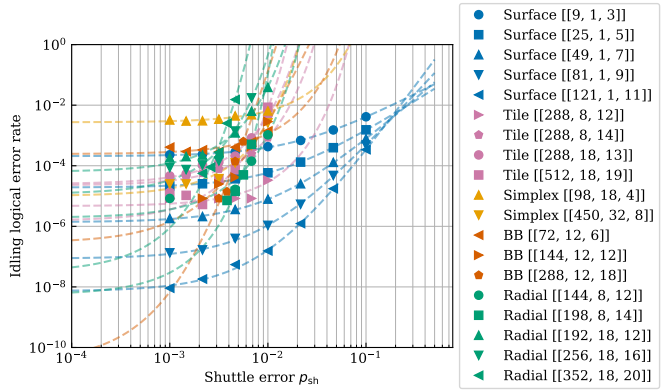

Fig. 10. Simulated logical error rate data and corresponding model fits for the code instances studied in this work, as a function of the shuttling error rate p_{sh}. All other error rates are kept constant (see Table II). Reducing the shuttling error rate below $p_{\mathrm{sh}} \approx 10^{-3}$ yields dimishing returns, as the codes are generally limited by the constant gate error rates.

In Figure 11, we plot the logical error rates (extrapolated where necessary) at specific shuttling error rates $10^{-2}, 3 \times 10^{-3}$, and 10^{-3}. The x axis shows the number of logical qubits that are encoded per 1000 physical qubits (calculated as $1000k/n$). We connect the surface code points with a dashed line to form a frontier and shade the area above. The unshaded region therefore corresponds to the area of net improvement over surface codes. A QLDPC data point in the unshaded region can therefore be viewed in one of two ways. Comparing to the surface code frontier above it, the QLDPC code can be seen as improving the logical error rate at an equivalent error rate. Alternatively, comparing to the surface code frontier to the left, the QLDPC code can be seen as improving the encoding rate at the same logical error rate. For $p_{\mathrm{sh}} = 10^{-2}$, we see that no QLDPC codes are able to improve upon the surface code. However, at $p_{\mathrm{sh}} = 3 \times 10^{-3}$, several BB and Radial codes enter the successful regime, and further improvements are obtained at $p_{\mathrm{sh}} = 10^{-3}$. Taking the example of the $[[144, 12, 12]]$ BB code at $p_{\mathrm{sh}} = 10^{-3}$, we can see that at the same logical error rate it improves upon the surface code encoding rate by approximately $4\times$, from 20 to 83; alternatively, at the same encoding rate, it reduces the logical error rate per logical qubit by two orders of magnitude, from 10^{-4} to 10^{-6}. We emphasize that these results are only possible with the circuit tailoring technique. Without circuit tailoring, no QLDPC codes outperform the surface code even

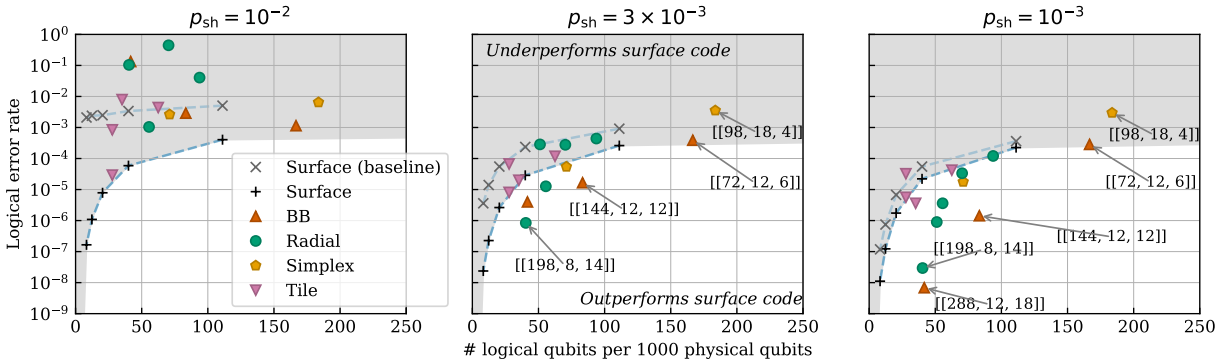

Fig. 11. Comparing CAbLECAR-optimized QLDPC and surface code performance. X axis shows the number of logical qubits that can be encoded in 1000 physical qubits (equivalent to $1000k/n$), and Y axis shows the idling logical error rate per logical qubit. Each plot shows a different value of p_{sh}; all other physical error rates are held constant. Surface code points are connected by a dashed line, and the unshaded region below the line indicates the regime where QLDPC codes outperform the CAbLECAR-optimized surface code in either encoding efficiency or logical protection. Also shown is the baseline surface code, which does not include circuit tailoring (Section III). For $p_{\text{sh}} = 10^{-2}$ (top), no studied QLDPC code outperforms the CAbLECAR surface code, but at $p_{\text{sh}} = 3 \times 10^{-3}$ (center) we identify several codes that provide up to 10× reduced logical error rates compared to the surface codes with similar encoding rates. For $p_{\text{sh}} = 10^{-3}$ (bottom), the improvements are much stronger, with the $[[288, 12, 18]]$ BB code providing approximately 3200× improvement in logical error rate compared to the surface code with similar encoding rate, and a 5× improvement in encoding rate compared to the surface code with similar logical error rate.

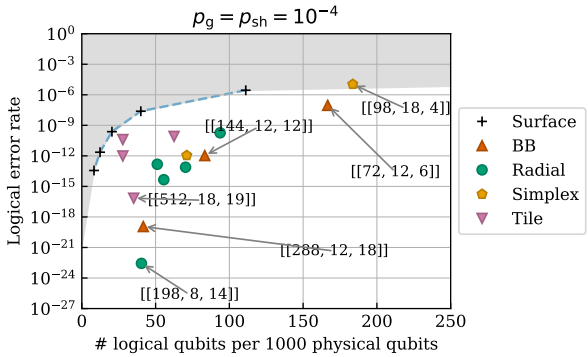

Fig. 12. Lowering physical gate and shuttling error rates to 10^{-4} enables strong improvements in performance, dramatically lowering logical error rates.

at $p_{\text{sh}} = 10^{-3}$ due to the accumulation of propagated errors to the data qubits.

Improving p_{sh} below 10^{-3} does not significantly improve the performance of most codes because the constant gate error rates become the limiting factor. In Figure 12, we investigate the case where p_{CX} and p_H are reduced to 10^{-4}. This enables very strong performance at lower shuttling error rates. For $p_{\text{sh}} = 10^{-4}$, we see strong gains for many of the studied codes. The $[[288, 12, 18]]$ BB code achieves approximately 8 orders of magnitude reduction in logical error rate compared to the surface code with the same encoding rate. However, reaching such low gate and shuttle error rates is hypothetical; we show these results for completeness, but in practice these low noise levels may not be reachable on a scaled-up physical device.

VI. Discussion

In this work, we introduced CAbLECAR, a comprehensive framework for efficiently scheduling and implementing high-rate QLDPC codes on shuttling-based semiconductor spin qubit architectures. By addressing both the physical noise mechanisms of shuttling and the algorithmic challenges of ancilla routing, we bridge the gap between near-term hardware capabilities and the non-local connectivity requirements of advanced quantum error correction.

First, we demonstrated that a simple, noise-aware circuit tailoring technique—appending Hadamard gates around shuttling operations for X ancilla qubits—effectively mitigates the dominant dephasing noise inherent to spin qubit shuttling. This prevents error propagation to the data qubits, relaxing the viable shuttling error threshold by 5-10× and extending the feasible range of ancilla movement.

Building upon this increased resilience, we developed Q-SIPP, a novel path-planning algorithm adapted from classical robotics to navigate the dynamic obstacle space of mobile ancilla qubits. Q-SIPP achieves near-ideal scheduling with only a 12.5% overhead compared to collision-free ideals. Furthermore, it significantly outperforms manually designed lockstep schedules, reducing shuttling overhead by up to 86% for code families with complex edge boundaries such as bivariate bicycle codes.

Through detailed circuit-level simulations, we identified the crossover regime in which these optimizations allow QLDPC codes to definitively surpass the surface code on this architecture. At shuttling error rates of $p_{\text{sh}} \leq 3 \times 10^{-3}$, families such as the bivariate bicycle and radial codes begin to offer drastic improvements. Notably, at $p_{\text{sh}} = 10^{-3}$, the [[144,12,12]] bivariate bicycle code achieves a 4× higher encoding rate or a 100× reduction in logical error rate compared to an equivalent surface code.

There are several promising avenues for future work. As noted in Section V, our evaluation utilized simple grid layouts for data qubits; explicitly co-optimizing the spatial mapping of

data qubits with the CAbLECAR routing engine could further minimize shuttling bottlenecks. Additionally, while the exact TSP solver within Q-SIPP's heuristic is highly efficient for the check weights evaluated in this work (up to weight 8), integrating approximate TSP solvers or designing alternative Q-SIPP heuristic functions will be necessary to schedule codes with significantly higher check weights. Finally, applying a modified Q-SIPP algorithm to heterogeneous chip layouts, such as architectures with dense data clusters connected by sparse long-range shuttling channels, remains an exciting direction.

Ultimately, our results indicate that the lack of native long-range interactions in spin qubits is not a fundamental roadblock for high-efficiency quantum error correction. With coordinated, collision-aware scheduling and tailored circuits, shuttling-based architectures present a highly scalable and viable path toward large-scale fault-tolerant quantum computation.

REFERENCES

[1] A. Yu. Kitaev. Quantum Error Correction with Imperfect Gates. In O. Hirota, A. S. Holevo, and C. M. Caves, editors, Quantum Communication, Computing, and Measurement, pages 181–188. Springer US, Boston, MA, 1997.

[2] Eric Dennis, Alexei Kitaev, Andrew Landahl, and John Preskill. Topological quantum memory. Journal of Mathematical Physics, 43(9):4452–4505, September 2002.

[3] Austin G. Fowler, Matteo Mariantoni, John M. Martinis, and Andrew N. Cleland. Surface codes: Towards practical large-scale quantum computation. Physical Review A, 86(3):032324, September 2012.

[4] Dominic Horsman, Austin G. Fowler, Simon Devitt, and Rodney Van Meter. Surface code quantum computing by lattice surgery. New Journal of Physics, 14(12):123011, December 2012.

[5] Sergey Bravyi, Andrew W. Cross, Jay M. Gambetta, Dmitri Maslov, Patrick Rall, and Theodore J. Yoder. High-threshold and low-overhead fault-tolerant quantum memory. Nature, 627(8005):778–782, March 2024.

[6] Thomas R. Scruby, Timo Hillmann, and Joschka Roffe. High-threshold, low-overhead and single-shot decodable fault-tolerant quantum memory, June 2024. arXiv:2406.14445 [quant-ph].

[7] Vincent Steffan, Shin Ho Choe, Nikolas P. Breuckmann, Francisco Revson Fernandes Pereira, and Jens Niklas Eberhardt. Tile Codes: High-Efficiency Quantum Codes on a Lattice with Boundary, April 2025. arXiv:2504.09171 [quant-ph].

[8] Ryota Nakai and Hayato Goto. Subsystem many-hypercube codes: High-rate concatenated codes with low-weight syndrome measurements. Physical Review Applied, 25(1):014032, January 2026.

[9] Andrew Cross, Zhiyang He, Patrick Rall, and Theodore Yoder. Linear-Size Ancilla Systems for Logical Measurements in QLDPC Codes, July 2024. arXiv:2407.18393 [quant-ph].

[10] Qian Xu, Hengyun Zhou, Guo Zheng, Dolev Bluvstein, J. Pablo Bonilla Ataides, Mikhail D. Lukin, and Liang Jiang. Fast and Parallelizable Logical Computation with Homological Product Codes, July 2024. arXiv:2407.18490 [quant-ph].

[11] Alexander Cowtan, Zhiyang He, Dominic J. Williamson, and Theodore J. Yoder. Parallel Logical Measurements via Quantum Code Surgery, March 2025. arXiv:2503.05003 [quant-ph].

[12] Esha Swaroop, Tomas Jochym-O'Connor, and Theodore J. Yoder. Universal adapters between quantum LDPC codes, March 2025. arXiv:2410.03628 [quant-ph].

[13] Hasan Sayginel, Stergios Koutsioumpas, Mark Webster, Abhishek Rajput, and Dan E. Browne. Fault-Tolerant Logical Clifford Gates from Code Automorphisms, May 2025. arXiv:2409.18175 [quant-ph].

[14] Guo Zhang, Yuanye Zhu, Xiao Yuan, and Ying Li. Constant-Overhead Magic State Injection into qLDPC Codes with Error Independence Guarantees, May 2025. arXiv:2505.06981 [quant-ph].

[15] Alexander J. Malcolm, Andrew N. Glaudell, Patricio Fuentes, Daryus Chandra, Alexis Schotte, Colby DeLisle, Rafael Haenel, Amir Ebrahimi, Joschka Roffe, Armanda O. Quintavalle, Stefanie J. Beale, Nicholas R. Lee-Hone, and Stephanie Simmons. Computing Efficiently in QLDPC Codes, August 2025. arXiv:2502.07150 [quant-ph].

[16] Qian Xu, Hengyun Zhou, Dolev Bluvstein, Madelyn Cain, Marcin Kalinowski, John Preskill, Mikhail D. Lukin, and Nishad Maskara. Batched high-rate logical operations for quantum LDPC codes, October 2025. arXiv:2510.06159 [quant-ph].

[17] Laura Pecorari, Francesco Paolo Guerci, Hugo Perrin, and Guido Pupillo. Addressable gate-based logical computation with quantum LDPC codes, November 2025. arXiv:2511.06124 [quant-ph].

[18] Willers Yang, Jason Chadwick, Mariesa H. Teo, Joshua Viszlai, and Fred Chong. Spacetime-Efficient and Hardware-Compatible Complex Quantum Logic Units in qLDPC Codes, March 2026. arXiv:2602.14273 [quant-ph].

[19] R. Maurand, X. Jehl, D. Kotekar-Patil, A. Corna, H. Bohuslavskyi, R. Laviéville, L. Hutin, S. Barraud, M. Vinet, M. Sanquer, and S. De Franceschi. A CMOS silicon spin qubit. Nature Communications, 7(1):13575, November 2016.

[20] M. Veldhorst, H. G. J. Eenink, C. H. Yang, and A. S. Dzurak. Silicon CMOS architecture for a spin-based quantum computer. Nature Communications, 8(1):1766, December 2017.

[21] Ruoyu Li, Luca Petit, David P. Franke, Juan Pablo Dehollain, Jonas Helsen, Mark Steudtner, Nicole K. Thomas, Zachary R. Yoscovits, Kanwal J. Singh, Stephanie Wehner, Lieven M. K. Vandersypen, James S. Clarke, and Menno Veldhorst. A crossbar network for silicon quantum dot qubits. Science Advances, 4(7):eaar3960, July 2018.

[22] Xiao Xue, Bishnu Patra, Jeroen P. G. van Dijk, Nodar Samkharadze, Sushil Subramanian, Andrea Corna, Brian Paquelet Wuetz, Charles Jeon, Farhana Sheikh, Esdras Juarez-Hernandez, Brando Perez Esparza, Huzaifa Rampurawala, Brent Carlton, Surej Ravikumar, Carlos Nieva, Sungwon Kim, Hyung-Jin Lee, Amir Sammak, Giordano Scappucci, Menno Veldhorst, Fabio Sebastiano, Masoud Babaie, Stefano Pellerano, Edoardo Charbon, and Lieven M. K. Vandersypen. CMOS-based cryogenic control of silicon quantum circuits. Nature, 593(7858):205–210, May 2021.

[23] Samuel Neyens, Otto K. Zietz, Thomas F. Watson, Florian Luthi, Aditi Nethwewala, Hubert C. George, Eric Henry, Mohammad Islam, Andrew J. Wagner, Felix Borjans, Elliot J. Connors, J. Corrigan, Matthew J. Curry, Daniel Keith, Roza Kotlyar, Lester F. Lampert, Mateusz T. Mądzik, Kent Millard, Fahd A. Mohiyaddin, Stefano Pellerano, Ravi Pillarisetty, Mick Ramsey, Rostyslav Savytskyy, Simon Schaal, Guoji Zheng, Joshua Ziegler, Nathaniel C. Bishop, Stephanie Bojarski, Jeanette Roberts, and James S. Clarke. Probing single electrons across 300-mm spin qubit wafers. Nature, 629(8010):80–85, May 2024.

[24] Paul Steinacker, Nard Dumoulin Stuyck, Wee Han Lim, Tuomo Tanttu, MengKe Feng, Andreas Nick, Santiago Serrano, Marco Candido, Jesus D. Cifuentes, Fay E. Hudson, Kok Wai Chan, Stefan Kubicek, Julien Jussot, Yann Canvel, Sofie Beyne, Yosuke Shimura, Roger Loo, Clement Godfrin, Bart Raes, Sylvain Baudot, Danny Wan, Arne Laucht, Chih Hwan Yang, Andre Saraiva, Christopher C. Escott, Kristiaan De Greve, and Andrew S. Dzurak. A 300 mm foundry silicon spin qubit unit cell exceeding 99% fidelity in all operations, October 2024. arXiv:2410.15590.

[25] Francesco Borsoi, Nico W. Hendrickx, Valentin John, Marcel Meyer, Sayr Motz, Floor van Riggelen, Amir Sammak, Sander L. de Snoo, Giordano Scappucci, and Menno Veldhorst. Shared control of a 16 semiconductor quantum dot crossbar array. Nature Nanotechnology, 19(1):21–27, January 2024.

[26] Hubert C. George, Mateusz T. Mądzik, Eric M. Henry, Andrew J. Wagner, Mohammad M. Islam, Felix Borjans, Elliot J. Connors, J. Corrigan, Matthew Curry, Michael K. Harper, Daniel Keith, Lester Lampert, Florian Luthi, Fahd A. Mohiyaddin, Sandra Murcia, Rohit Nair, Rambert Nahm, Aditi Nethwewala, Samuel Neyens, Bishnu Patra, Roy D. Raharjo, Carly Rogan, Rostyslav Savytskyy, Thomas F. Watson, Josh Ziegler, Otto K. Zietz, Stefano Pellerano, Ravi Pillarisetty, Nathaniel C. Bishop, Stephanie A. Bojarski, Jeanette Roberts, and James S. Clarke. 12-Spin-Qubit Arrays Fabricated on a 300 mm Semiconductor Manufacturing Line. Nano Letters, 25(2):793–799, January 2025.

[27] Sieu D. Ha, Edwin Acuna, Kate Raach, Zachery T. Bloom, Teresa L. Brecht, James M. Chappell, Maxwell D. Choi, Justin E. Christensen, Ian T. Counts, Dominic Daprano, J. P. Dodson, Kevin Eng, David J. Fialkow, Christina A. C. Garcia, Wonill Ha, Thomas R. B. Harris,

nathan holman, Isaac Khalaf, Justine W. Matten, Christi A. Peterson, Clifford E. Plesha, Matthew J. Ruiz, Aaron Smith, Bryan J. Thomas, Samuel J. Whiteley, Thaddeus D. Ladd, Michael P. Jura, Matthew T. Rakher, and Matthew G. Borselli. Two-dimensional Si spin qubit arrays with multilevel interconnects, February 2025. arXiv:2502.08861 [quant-ph].

[28] I. Fernández de Fuentes, E. Raymenants, B. Undseth, O. Pietx-Casas, S. Philips, M. Mądzik, S.L. de Snoo, S.V. Amitonov, L. Tryputen, A.T. Schmitz, A.Y. Matsuura, G. Scappucci, and L.M.K. Vandersypen. Running a Six-Qubit Quantum Circuit on a Silicon Spin-Qubit Array. PRX Quantum, 7(1):010308, January 2026.

[29] K. Takeda, A. Noiri, J. Yoneda, T. Nakajima, and S. Tarucha. Resonantly Driven Singlet-Triplet Spin Qubit in Silicon. Physical Review Letters, 124(11):117701, March 2020.

[30] Adam R. Mills, Charles R. Guinn, Michael J. Gullans, Anthony J. Sigillito, Mayer M. Feldman, Erik Nielsen, and Jason R. Petta. Two-qubit silicon quantum processor with operation fidelity exceeding 99%. Science Advances, 8(14):eabn5130, April 2022.

[31] Aaron J. Weinstein, Matthew D. Reed, Aaron M. Jones, Reed W. Andrews, David Barnes, Jacob Z. Blumoff, Larken E. Euliss, Kevin Eng, Bryan H. Fong, Sieu D. Ha, Daniel R. Hulbert, Clayton A. C. Jackson, Michael Jura, Tyler E. Keating, Joseph Kerckhoff, Andrey A. Kiselev, Justine Matten, Golam Sabbir, Aaron Smith, Jeffrey Wright, Matthew T. Rakher, Thaddeus D. Ladd, and Matthew G. Borselli. Universal logic with encoded spin qubits in silicon. Nature, 615(7954):817–822, March 2023.

[32] Yi-Hsien Wu, Leon C. Camenzind, Patrick Bütler, Ik Kyeong Jin, Akito Noiri, Kenta Takeda, Takashi Nakajima, Takashi Kobayashi, Giordano Scappucci, Hsi-Sheng Goan, and Seigo Tarucha. Simultaneous High-Fidelity Single-Qubit Gates in a Spin Qubit Array, July 2025. arXiv:2507.11918 [quant-ph].

[33] Joseph D. Broz, Jesse C. Hoke, Edwin Acuna, and Jason R. Petta. Demonstration of an always-on exchange-only spin qubit, August 2025. arXiv:2508.01033 [quant-ph].

[34] Paul Steinacker, Nard Dumoulin Stuyck, Wee Han Lim, Tuomo Tanttu, MengKe Feng, Santiago Serrano, Andreas Nickl, Marco Candido, Jesus D. Cifuentes, Ensar Vahapoglu, Samuel K. Bartee, Fay E. Hudson, Kok Wai Chan, Stefan Kubicek, Julien Jussot, Yann Canvel, Sofie Beyne, Yosuke Shimura, Roger Loo, Clement Godfrin, Bart Raes, Sylvain Baudot, Danny Wan, Arne Laucht, Chih Hwan Yang, Andre Saraiva, Christopher C. Escott, Kristiaan De Greve, and Andrew S. Dzurak. Industry-compatible silicon spin-qubit unit cells exceeding 99% fidelity. Nature, 646(8083):81–87, October 2025.

[35] Mateusz T. Mądzik, Florian Luthi, Gian Giacomo Guerreschi, Fahd A. Mohiyaddin, Felix Borjans, Jason D. Chadwick, Matthew J. Curry, Joshua Ziegler, Sarah Atanasov, Peter L. Bavdaz, Elliot J. Connors, J. Corrigan, H. Ekmel Ercan, Robert Flory, Hubert C. George, Benjamin Harpt, Eric Henry, Mohammad M. Islam, Nader Khammassi, Daniel Keith, Lester F. Lampert, Todor M. Mladenov, Randy W. Morris, Aditi Nethwewala, Samuel Neyens, René Otten, Linda P. Osuna Ibarra, Bishnu Patra, Ravi Pillarisetty, Shavindra Premaratne, Mick Ramsey, Andrew Risinger, John D. Rooney, Rostyslav Savytskyy, Thomas F. Watson, Otto K. Zietz, Anne Y. Matsuura, Stefano Pellerano, Nathaniel C. Bishop, Jeanette Roberts, and James S. Clarke. Operating two exchange-only qubits in parallel. Nature, 647(8091):870–875, November 2025.

[36] Jason D. Chadwick, Willers Yang, Joshua Viszlai, and Frederic T. Chong. A manufacturable surface code architecture for spin qubits with fast transversal logic, March 2026. arXiv:2512.07131 [quant-ph].

[37] Jelmer M. Boter, Juan P. Dehollain, Jeroen P.G. Van Dijk, Yuanxing Xu, Toivo Hensgens, Richard Versluis, Henricus W.L. Naus, James S. Clarke, Menno Veldhorst, Fabio Sebastiano, and Lieven M.K. Vandersypen. Spiderweb Array: A Sparse Spin-Qubit Array. Physical Review Applied, 18(2):024053, August 2022.

[38] Matthias Künne, Alexander Willmes, Max Oberländer, Christian Gorjaew, Julian D. Teske, Harsh Bhardwaj, Max Beer, Eugen Kammerloher, René Otten, Inga Seidler, Ran Xue, Lars R. Schreiber, and Hendrik Bluhm. The SpinBus architecture for scaling spin qubits with electron shuttling. Nature Communications, 15(1):4977, June 2024.

[39] Rubén M. Otxoa, Josu Etxezarreta Martinez, Paul Schnabl, Normann Mertig, Charles Smith, and Frederico Martins. SpinHex: A low-crosstalk, spin-qubit architecture based on multi-electron couplers, April 2025. arXiv:2504.03149 [quant-ph].

[40] Berat Yenilen, Arnau Sala, Hendrik Bluhm, Markus Müller, and Manuel Rispler. Performance of the spin qubit shuttling architecture for a surface code implementation, March 2025. arXiv:2503.10601 [quant-ph].

[41] Mike Phillips and Maxim Likhachev. SIPP: Safe interval path planning for dynamic environments. In 2011 IEEE International Conference on Robotics and Automation, pages 5628–5635, Shanghai, China, May 2011. IEEE.

[42] Eric Henry. 2D Silicon Spin Qubits. In APS March Meeting 2025, MAR-C19:8. American Physical Society, March 2025.

[43] Alexei M. Tyryshkin, Shinichi Tojo, John J. L. Morton, Helge Riemann, Nikolai V. Abrosimov, Peter Becker, Hans-Joachim Pohl, Thomas Schenkel, Michael L. W. Thewalt, Kohei M. Itoh, and S. A. Lyon. Electron spin coherence exceeding seconds in high-purity silicon. Nature Materials, 11(2):143–147, February 2012.

[44] M. Veldhorst, J. C. C. Hwang, C. H. Yang, A. W. Leenstra, B. De Ronde, J. P. Dehollain, J. T. Muhonen, F. E. Hudson, K. M. Itoh, A. Morello, and A. S. Dzurak. An addressable quantum dot qubit with fault-tolerant control-fidelity. Nature Nanotechnology, 9(12):981–985, December 2014.

[45] Guido Burkard, Thaddeus D. Ladd, Andrew Pan, John M. Nichol, and Jason R. Petta. Semiconductor spin qubits. Reviews of Modern Physics, 95(2):025003, June 2023.

[46] Brennan Undseth, Nicola Meggiato, Yi-Hsien Wu, Sam R. Katiraee-Far, Larysa Tryputen, Sander L. de Snoo, Davide Degli Esposti, Giordano Scappucci, Eliška Greplová, and Lieven M. K. Vandersypen. Weight-four parity checks with silicon spin qubits, January 2026. arXiv:2601.23267 [cond-mat].

[47] Inga Seidler, Tom Struck, Ran Xue, Niels Focke, Stefan Trellenkamp, Hendrik Bluhm, and Lars R. Schreiber. Conveyor-mode single-electron shuttling in Si/SiGe for a scalable quantum computing architecture. npj Quantum Information, 8(1):100, August 2022.

[48] A.M.J. Zwerver, S.V. Amitonov, S.L. De Snoo, M.T. Mądzik, M. Rimbach-Russ, A. Sammak, G. Scappucci, and L.M.K. Vandersypen. Shuttling an Electron Spin through a Silicon Quantum Dot Array. PRX Quantum, 4(3):030303, July 2023.

[49] Ran Xue, Max Beer, Inga Seidler, Simon Humpohl, Jhih-Sian Tu, Stefan Trellenkamp, Tom Struck, Hendrik Bluhm, and Lars R. Schreiber. Si/SiGe QuBus for single electron information-processing devices with memory and micron-scale connectivity function. Nature Communications, 15(1):2296, March 2024.

[50] Maxim De Smet, Yuta Matsumoto, Anne-Marije J. Zwerver, Larysa Tryputen, Sander L. de Snoo, Sergey V. Amitonov, Amir Sammak, Nodar Samkharadze, Önder Gül, Rick N. M. Wasserman, Maximilian Rimbach-Russ, Giordano Scappucci, and Lieven M. K. Vandersypen. High-fidelity single-spin shuttling in silicon. June 2024. arXiv:2406.07267 [cond-mat].

[51] Yasuo Oda, Merritt P. Losert, and J. P. Kestner. Suppressing Si Valley Excitation and Valley-Induced Spin Dephasing for Long-Distance Shuttling. Physical Review Letters, 136(2):020802, January 2026.

[52] Natalie D. Foster, Jacob D. Henshaw, Martin Rudolph, Dwight R. Luhman, and Ryan M. Jock. Dephasing and error dynamics affecting a singlet-triplet qubit during coherent spin shuttling. npj Quantum Information, 11(1):63, April 2025.

[53] Zhenyu Cai, Adam Siegel, and Simon Benjamin. Looped Pipelines Enabling Effective 3D Qubit Lattices in a Strictly 2D Device. PRX Quantum, 4(2):020345, June 2023.

[54] Joshua Viszlai, Willers Yang, Sophia Fuhui Lin, Junyu Liu, Natalia Nottingham, Jonathan M. Baker, and Frederic T. Chong. Matching Generalized-Bicycle Codes to Neutral Atoms for Low-Overhead Fault-Tolerance, November 2023. arXiv:2311.16980 [quant-ph].

[55] Qian Xu, J. Pablo Bonilla Ataides, Christopher A. Pattison, Nithin Raveendran, Dolev Bluvstein, Jonathan Wurtz, Bane Vasic, Mikhail D. Lukin, Liang Jiang, and Hengyun Zhou. Constant-Overhead Fault-Tolerant Quantum Computation with Reconfigurable Atom Arrays. April 2024. arXiv:2308.08648 [quant-ph].

[56] Adam Siegel, Armands Strikis, and Michael Fogarty. Towards Early Fault Tolerance on a $2\ifmmode\times\else\texttimes\fi{}N$ Array of Qubits Equipped with Shuttling. PRX Quantum, 5(4):040328, November 2024.

[57] Pengyu Liu, Mingkuan Xu, Hengyun Zhou, Hanrui Wang, Umut A. Acar, and Yunong Shi. ConiQ: Enabling Concatenated Quantum Error Correction on Neutral Atom Arrays, August 2025. arXiv:2508.05779 [cs].

[58] Argyris Giannisis Manes and Jahan Claes. Distance-preserving stabilizer measurements in hypergraph product codes. Quantum, 9:1618, January 2025. arXiv:2308.15520 [quant-ph].
[59] H. Bombin and M. A. Martin-Delgado. Optimal resources for topological two-dimensional stabilizer codes: Comparative study. Physical Review A, 76(1):012305, July 2007.
[60] Craig Gidney. Stim: a fast stabilizer circuit simulator. Quantum, 5:497, July 2021. arXiv:2103.02202 [quant-ph].